🔍 Observability

Logging, Metrics & Tracing — Module 9 of 13

Enterprise EKS Platform

📋 Agenda

📚 Concepts

- Three Pillars of Observability
- Container Logging Model
- Kubernetes Metrics Architecture
- Distributed Tracing Fundamentals
- SLOs and Error Budgets

🛠️ Hands-On

- kubectl logs and centralized logging
- Prometheus & Grafana stack
- OpenTelemetry instrumentation
- Alertmanager configuration
- CloudWatch Container Insights

Three Pillars of Observability

Logs, Metrics, and Traces

🤔 Observability vs. Monitoring

Monitoring

- Predefined dashboards and alerts
- Answers **known** questions
- "Is CPU above 80%?"
- Reactive: tells you *what* is broken

Observability

- Explore system state dynamically
- Answers **unknown** questions
- "Why are requests from region X slow?"
- Proactive: tells you *why* it's broken

Goal: Build systems where internal state is externally queryable without deploying new code.

📈 The Three Pillars

📜 Logs

- What happened
- Immutable records
- High cardinality

📊 Metrics

- How much / how fast
- Aggregatable
- Low storage cost

🔗 Traces

- Where time is spent
- Causal relationships
- Cross-service visibility

🔄 Pillars Working Together

Typical Debugging Flow

1. **Metrics** alert fires — error rate exceeds threshold
2. **Traces** reveal which service in the call chain is slow
3. **Logs** from that service show the root cause (e.g., DB timeout)

> **Correlation is key:** Use trace IDs to link logs, metrics, and traces for the same request.

Logging in Kubernetes

Container stdout/stderr to centralized stores

📦 Container Logging Model

How Kubernetes Captures Logs

- Containers write to `stdout` and `stderr`
- Container runtime redirects streams to a logging driver
- Kubelet stores logs at `/var/log/containers/`
- Log rotation managed by kubelet (`containerLogMaxSize`, `containerLogMaxFiles`)
- Logs are **ephemeral** — lost when pod is deleted

> **Key implication:** Never rely on local container logs. Ship them to a durable store.

💻 kubectl logs

```
# Follow logs in real-time
kubectl logs -f deployment/api-server

# View logs from a previous container instance (after restart)
kubectl logs pod/api-server-7d4f8b --previous

# Logs from a specific container in a multi-container pod
kubectl logs pod/api-server-7d4f8b -c sidecar-proxy

# Logs from all pods matching a label
kubectl logs -l app=api-server --all-containers=true

# Last 100 lines with timestamps
kubectl logs deployment/api-server --tail=100 --timestamps

# Logs from the last 30 minutes
kubectl logs deployment/api-server --since=30m
```

> **Tip:** Use `stern` for multi-pod log tailing with color-coded output per pod.

🏗️ Centralized Logging Architecture

DaemonSet-based Collection Pattern

Node agent reads log files → enriches with K8s metadata → ships to backend

📦 Collectors

- **Fluent Bit** — lightweight, C-based
- **Fluentd** — flexible, plugin-rich
- **Vector** — high-performance, Rust-based
- **CloudWatch Agent** — AWS-

🖨️ Backends

- **Elasticsearch + Kibana** (ELK/EFK)
- **Grafana Loki** — like Prometheus for logs
- **AWS CloudWatch Logs**
- **Datadog / Splunk**

🔧 Fluent Bit on EKS

```
# Fluent Bit DaemonSet — key container and volume config
kind: DaemonSet
# ... metadata, spec.selector ...
    spec:
      containers:
      - name: fluent-bit
        image: amazon/aws-for-fluent-bit:2.31.12
        volumeMounts:
        - { name: varlog, mountPath: /var/log }
        - { name: config, mountPath: /fluent-bit/etc/ }
      volumes:
      - { name: varlog, hostPath: { path: /var/log } }
      - { name: config, configMap: { name: fluent-bit-config } }
```

📝 Structured Logging Best Practices

❌ Unstructured

```
2024-03-15 ERROR: Failed to
connect to database host
db-primary port 5432 timeout
after 30s for user api-svc
```

✅ Structured (JSON)

```
{
  "timestamp": "2024-03-15T10:23:45Z",
  "level": "error",
  "message": "database connection failed",
  "host": "db-primary",
  "port": 5432,
  "timeout_seconds": 30,
  "service": "api-svc",
  "trace_id": "abc123def456"
}
```

Why structured? Enables filtering, aggregation, and correlation. Always include `trace_id` for cross-pillar linkage.

☁️ CloudWatch Logs with EKS

EKS Log Groups Structure

- `/aws/eks/cluster-name/cluster` — control plane logs
- `/aws/containerinsights/cluster-name/application` — pod logs
- `/aws/containerinsights/cluster-name/host` — node-level logs
- `/aws/containerinsights/cluster-name/dataplane` — kubelet, kube-proxy

CloudWatch Logs Insights query:

```
fields @timestamp, @message, kubernetes.pod_name
| filter @message like /error/i
| sort @timestamp desc
| limit 50
```

🔬 Logging to Splunk on EKS

Fluent Bit → Splunk HEC Pipeline

Fluent Bit ships logs to Splunk via the `splunk` output plugin and HTTP Event Collector (HEC)

```
# fluent-bit OUTPUT section
[OUTPUT]
    Name           splunk
    Match          kube.*
    Host           splunk-hec.internal.example.com
    Port           8088
    Splunk_Token   ${SPLUNK_HEC_TOKEN}
    TLS            On
    TLS.Verify     On
    Splunk_Send_Raw On
    # index, source, sourcetype set per-namespace
    Event_Index    k8s_prod
    Event_Sourcetype _json
    Event_Source   fluent-bit
```

Key detail: HEC token is stored in a Kubernetes Secret and injected via `envFrom`. Index-per-namespace routing uses Fluent Bit's `Rewrite_Tag` filter to direct logs to the correct Splunk index.

Metrics in Kubernetes

Measuring system health and performance

🏗️ Kubernetes Metrics Architecture

📏 Resource Metrics

- CPU & memory per pod/node
- Powers HPA & VPA
- Lightweight, in-memory

🎯 Custom Metrics

- App-specific (queue depth, etc.)
- Prometheus Adapter
- Enables custom HPA scaling

🌍 External Metrics

- Outside-cluster sources
- SQS queue length, etc.
- CloudWatch metrics adapter

🔥 Prometheus Fundamentals

Pull-Based Architecture

- Prometheus **scrapes** /metrics endpoints on a schedule
- Service discovery via Kubernetes API (pods, services, endpoints)
- Time-series database with PromQL query language
- Built-in alerting via Alertmanager integration

Four metric types: Counter (only goes up), Gauge (arbitrary values), Histogram (bucketed distributions), Summary (quantile calculations)

🔭 ServiceMonitor Configuration

```yaml
# ServiceMonitor — Prometheus Operator CRD
kind: ServiceMonitor
# ... metadata with release: prometheus label ...
spec:
  selector:
    matchLabels: { app: api-server }
  endpoints:
  - port: metrics
    path: /metrics
    interval: 15s
```

ServiceMonitor is a CRD from the Prometheus Operator. It declaratively defines scrape targets instead of editing Prometheus config directly.

🔎 PromQL Essentials

```
# Request rate per second over 5 minutes
rate(http_requests_total{service="api"}[5m])

# 99th percentile latency
histogram_quantile(0.99,
  rate(http_request_duration_seconds_bucket{service="api"}[5m]))

# Error rate percentage
sum(rate(http_requests_total{status=~"5.."}[5m]))
  /
sum(rate(http_requests_total[5m])) * 100

# CPU usage by namespace
sum by (namespace) (
  rate(container_cpu_usage_seconds_total[5m])
)

# Memory usage vs. limits
container_memory_working_set_bytes / container_spec_memory_limit_bytes
```

📊 Grafana Dashboards

🎭 Essential Dashboards

- **Cluster Overview** — node health, capacity
- **Namespace Resources** — CPU, memory, network
- **Pod Details** — restarts, OOMKills, throttling
- **Persistent Volumes** — usage, IOPS
- **Networking** — DNS, ingress, service mesh

📋 USE & RED Methods

USE (Infrastructure):
Utilization, Saturation, Errors

RED (Services):
Rate, Errors, Duration

🔧 kube-state-metrics & node-exporter

⚙️ kube-state-metrics

Generates metrics from Kubernetes API objects

- Deployment replicas desired vs. available
- Pod phase, conditions, restarts
- Job/CronJob success/failure counts
- Resource requests and limits

💻 node-exporter

Exports hardware and OS metrics from nodes

- CPU, memory, disk, network I/O
- Filesystem usage and inodes
- System load averages
- Runs as DaemonSet on each node

Together: kube-state-metrics tells you what Kubernetes *thinks*; node-exporter tells you what the node *experiences*.

☁️ CloudWatch Container Insights

AWS-Native Observability for EKS

- Pre-built dashboards for clusters, nodes, pods, services
- Automatic collection of CPU, memory, disk, network metrics
- Deployed via **CloudWatch Agent** or **ADOT Collector** (DaemonSet)
- Integrated with CloudWatch Alarms and SNS notifications
- Enhanced observability mode adds Prometheus-compatible metrics

```
# Enable Container Insights with the ADOT add-on
aws eks create-addon --cluster-name mycompany-prod \
  --addon-name amazon-cloudwatch-observability \
  --addon-version v1.5.0-eksbuild.1
```

🔗 Distributed Tracing

Following requests across service boundaries

🔭 Tracing Concepts

Key Terminology

- **Trace** — end-to-end journey of a request
- **Span** — a single operation within a trace
- **Parent/Child** — spans form a tree structure
- **Context Propagation** — passing trace IDs across service boundaries

🕑 Anatomy of a Trace

```
Trace: abc-123
├─ Span: API Gateway    [0ms - 250ms]
│  ├─ Span: Auth Svc    [10ms - 40ms]
│  └─ Span: Order Svc   [50ms - 240ms]
│     ├─ Span: DB Query  [60ms - 120ms]
│     └─ Span: Cache     [130ms - 140ms]
```

🌐 OpenTelemetry (OTel)

📚 SDKs

- Auto-instrumentation
- Manual span creation
- Java, Python, Go, Node.js

📦 Collector

- Receives, processes, exports
- Pipeline: receivers → processors → exporters
- Deploy as sidecar or DaemonSet

🔌 OTLP

- Standard wire protocol
- gRPC and HTTP transport
- Covers logs, metrics, traces

🔧 OTel Collector Pipeline

```yaml
# OTel Collector config.yaml
receivers:
  otlp:
    protocols:
      grpc: { endpoint: 0.0.0.0:4317 }
      http: { endpoint: 0.0.0.0:4318 }
processors:
  batch:
    timeout: 5s
    send_batch_size: 1024
  memory_limiter:
    limit_mib: 512
exporters:
  awsxray: { region: us-east-2 }
  prometheus: { endpoint: 0.0.0.0:8889 }
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [awsxray]
```

🔭 Tracing Backends

🔎 Jaeger

- CNCF graduated project
- Full trace search and comparison
- Dependency graph visualization
- Supports Elasticsearch or Cassandra storage
- Self-hosted, full control

☁️ AWS X-Ray

- Managed service, no infrastructure
- Native AWS service integration
- Service map auto-generated
- Trace groups and filter expressions
- Integrates via ADOT Collector

Recommended pattern: Use X-Ray on EKS for AWS-native visibility. Add Jaeger for advanced trace analysis or self-hosted retention.

🚨 Alerting & Incident Response

From detection to resolution

🔔 Alertmanager Configuration

```yaml
# PrometheusRule — Prometheus Operator CRD
kind: PrometheusRule
# ... metadata with release: prometheus label ...
spec:
  groups:
  - name: api-server
    rules:
    - alert: HighErrorRate
      expr: |
        sum(rate(http_requests_total{status=~"5..",job="api"}[5m]))
        / sum(rate(http_requests_total{job="api"}[5m])) > 0.05
      for: 5m                    # prevents flapping
      labels: { severity: critical }  # drives routing
      annotations:
        summary: "API error rate above 5%"
        runbook_url: "https://wiki.internal/runbooks/api-errors"
```

Alert Routing

```
# Alertmanager configuration
route:
  receiver: default-slack
  group_by: [alertname, namespace]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
  # ... routing rules per severity ...
receivers:
- name: pagerduty-oncall
  pagerduty_configs:
  - service_key_file: /etc/alertmanager/secrets/pd-key
  # ... slack, email, and other receiver configs
```

Enterprise Alerting Destinations

Netcool (Webhook)

- Enterprise event management
- Alertmanager `webhook_configs`
- POST JSON to Netcool probe REST endpoint
- Maps severity → Netcool class

Email (SMTP)

- Built-in `email_configs` receiver
- Corporate SMTP relay
- HTML templates for context
- Best for low-urgency / ticket alerts

WebEx (Webhook)

- WebEx Teams Incoming Webhook
- Uses `webhook_configs`
- Adaptive Card payload
- Team Spaces for on-call visibility

Routing tree strategy: `severity: critical` → Netcool + WebEx | `severity: warning` → Email + WebEx | `severity: info` →

🎯 SLOs and Error Budgets

Definitions

- **SLI** — Service Level Indicator (measured metric)
- **SLO** — Service Level Objective (target value)
- **SLA** — Service Level Agreement (contractual)
- **Error Budget** — tolerable failure amount

📈 Example

Component	Value
SLI	% requests < 200ms
SLO	99.9% over 30 days
Error Budget	43.2 min/month

> Budget remaining: 28.7 min

🔥 Burn Rate Alerting

Multi-Window Burn Rate Strategy

Alert based on how fast you're consuming your error budget, not just on raw error rate

Severity	Burn Rate	Long Window	Short Window	Budget Consumed
Page	14.4x	1 hour	5 min	2% in 1 hour
Page	6x	6 hours	30 min	5% in 6 hours
Ticket	3x	1 day	2 hours	10% in 1 day
Ticket	1x	3 days	6 hours	10% in 3 days

🛠️ Systematic Debugging Workflow

Step-by-Step Approach

1. **Alert fires** — check severity, read runbook
2. **Metrics** — identify affected service, scope of impact
3. **Traces** — find slow/failing spans, isolate the service
4. **Logs** — search by trace ID for exact error details
5. **Kubernetes state** — `kubectl get events`, pod status, resource pressure
6. **Mitigate** — rollback, scale, or isolate, then fix root cause

> **Anti-pattern:** Jumping straight to `kubectl logs` without first narrowing the scope via metrics and traces.

kubectl Debugging Commands

```
# Check recent events for anomalies
kubectl get events --sort-by='.lastTimestamp' -n production

# Identify resource pressure on nodes
kubectl top nodes
kubectl top pods -n production --sort-by=cpu

# Check pod conditions and status
kubectl get pods -n production -o wide
kubectl describe pod <pod-name> -n production

# Check for OOMKilled containers
kubectl get pods -n production -o json | \
  jq '.items[] | select(.status.containerStatuses[]?
    .lastState.terminated.reason == "OOMKilled")
    | .metadata.name'

# Debug networking with ephemeral containers
kubectl debug -it <pod-name> --image=nicolaka/netshoot -- bash
```

📚 Module 9 Summary

📜 Data Collection

- Logs: stdout/stderr, Fluent Bit, CloudWatch
- Metrics: metrics-server, Prometheus, Container Insights
- Traces: OpenTelemetry, X-Ray, Jaeger
- Structured logging with trace IDs for correlation

🚨 Operational Response

- Alertmanager routes alerts by severity
- SLOs define measurable reliability targets
- Burn rate alerts catch budget consumption trends
- Systematic debugging: metrics → traces → logs

👨‍💻 Lab 9: Observability

Hands-On Exercise

- Explore kubectl logs flags and multi-container pod logging
- Deploy structured JSON logging and filter with jq
- Run PromQL queries for CPU, memory, and pod metrics
- Navigate Grafana dashboards to investigate workload resources
- Create a PrometheusRule alert for pod restarts
- Debug a failing application using events, logs, and metrics

> **Duration:** ~45-55 minutes

🎯 Key Takeaways

1. Correlate across all three pillars. Use trace IDs to link logs, metrics, and traces. Without correlation, you have three separate silos.

2. Structured logging is essential. JSON logs with consistent fields enable automated parsing, filtering, and alerting.

3. Alert on SLOs, not symptoms. Burn rate alerting reduces noise and focuses on user-facing impact.

4. Instrument with OpenTelemetry. Vendor-neutral instrumentation decouples your code from any specific backend.

? Questions & Discussion

Module 9: Observability — Logging, Metrics & Tracing

Up Next: Module 10 — Health Probes & Application Debugging